

학습한 ML/DL 모델
판정 인코더 모델 명세

SKN Final Project · 3팀 「한끗」 — CopyLane 최종 산출물

모델 학습 **박수진** (인코더 담당 박수진 · 소성민)
잠금 채택 **오한빈** (팀장)
개정 **2026-10-21** (10-22 유형별 지표 보완) · 기준 커밋 **ksr b54246d**

제출용

목차

1. 한눈에 보기 4
2. 이 묶음의 구성 5
3. 베이스 모델과 구조 7
4. 라벨 스킴 8
5. 입력과 출력 9
6. 학습 데이터 10
7. 학습 설정 11
8. 기록된 성능 (모두 dev · 게이트 아님) 12
9. 파일과 sha256 14
10. 앱이 불러오는 방법 (코드 근거) 15
11. 판 이력 16
12. 함께 쓰이는 다른 모델 (목록만) 17
13. 한계와 남은 일 18

> 2차 제출본(09-28)은 노트북과 가중치 파일만 묶은 꾸러미였다. 이번 판은 같은 구성(노트북 + `학습결과 파일/`)을 지금 앱이 쓰는 모델로 바꾸고, 이 명세서를 새로 넣었다. 이 문서의 모든 수치는 **dev(개발 검증셋)** 에서 잰 값이다. 이 모델(D4-20)은 봉인 평가셋에서 잰 적이 없다 — 재분할 뒤의 새 봉인 평가셋 1,141 행은 열지 않았다. (이전 판 v10 · D4-10 은 10-14 에 재분할 전의 옛 봉인 2,489 행에서 인코더 단독으로 한 번 쟀다 — D4-T · 보고 전용.)

---

1. 한눈에 보기

항목	값	근거
모델 이름 (판 이름)	`copylane-encoder-kcbert-D4-20-기능성편입`	`models.lock.json` `roles.encoder.adopted`
역할	판정 인코더 — 광고 문장 하나와 품목을 받아 위반 유형 9가지의 확률을 낸다. 법 조항 판단은 하지 않는다(판정은 그래프의 법별 노드와 `judge` 가 한다)	`app/encoder.py:1-5` · D-131
종류	DL · 다중 라벨 문장 분류(파인튜닝)	`config.json` `problem_type = multi_label_classification`
베이스 모델	`beomi/kcbert-base` (한국어 BERT) · 라이선스 Apache-2.0	`label_scheme.json` `backbone` · `data_sources.yaml:2170-2174`
채택 결정	D-349 · 2026-10-19 잠금 채택(팀장이 `scripts.model_lock add … --as adopted --decision D-349` 실행)	`docs/00_설계결정기록.md` D-349 상태 줄
판 지문	`c451942436f0` (5개 파일의 이름·sha256 목록 전체의 sha256 앞 12자)	`app/model_lock.py:84-90` · 이 묶음 `학습기록/sha256_대조_2026-10-21.md`
학습	2026-10-18 · Google Colab T4 · 시드 48 · 49 · 50 세 번(약 13분) · 배포 시드 49	`docs/psj/reports/D4_실험기록_20261012.md:1104, 1119`
되돌림 판 (잠금에 등재)	D4-10 품목입력사실전제 → v10-카드17-검수정상음성 → v10-합성O-이유O	`models.lock.json` `roles.encoder.rollback`

2. 이 묶음의 구성

```
학습한 ML-DL 모델 (최종)/
├── copylane_encoder_D4-20_기능성편입_20261018.ipynb   ← 이 모델을 만든 학습 노트북(박수진)
├── 학습결과 파일/                                     ← 앱이 읽는 모델 폴더 그대로(5개 파일 · 잠금과 sha256 일치)
│   ├── config.json
│   ├── label_scheme.json
│   ├── model.safetensors
│   ├── tokenizer.json
│   └── tokenizer_config.json
├── 학습기록/
│   ├── D4-20_기능성편입_dev_20261018.md       ← dev 집계표(시드 평균 · 혼동표 · 지정 8행)
│   ├── D4_실험기록_D4-20절_발췌.md            ← 학습 전에 적은 합격 기준과 결과(원문 그대로)
│   ├── D4-20_유형별지표_확장dev_20261022.md   ← 유형별 정밀도 · 재현율 · F1 · accuracy(10-22 재측정 · §8-3)
│   ├── D4-20_유형별지표_확장dev_20261022.json ← 같은 측정의 수(문장 원문 없음)
│   └── sha256_대조_2026-10-21.md              ← 파일별 sha256 과 잠금 대조 결과
├── 채점_스크립트/                             ← 문턱 계산 · dev 채점(로컬)
│   ├── make_d415_scheme.py                    ← label_scheme.json(문턱 두 벌)과 배포 폴더를 만든다
│   ├── d4_20_functional.py                    ← D4-20 dev 판정(D4-18 ×2 와 비교)
│   ├── d4_20_type_metrics.py                  ← 유형별 지표(10-22 · 앱 로더 · 문턱으로 확장 dev 채점)
│   └── 그 밖 6개                              ← 위 두 스크립트가 불러 쓰는 모듈(아래 목록)
├── 모델_명세.md
└── 모델_명세.pdf
```

`채점_스크립트/` 의 나머지 모듈: `card17_adopt.py` · `score_encoder_dev.py` · `d4_04_product_input.py` · `d4_10_fact_train.py` · `d4_15_resplit.py` · `d4_17_ensemble_grid.py`. 모두 저장소 `docs/psj/e2e_prototype/` 에서 그대로 옮겼다 (`d4_20_type_metrics.py` 와 `학습기록/` 의 유형별 지표 두 파일은 10-22 박수진 브랜치 `78b0e49` 에서 옮김). `d4_15_resplit.py` 는 저장소의 `preprocess/encoder_resplit.py` 를 불러 쓰므로 저장소 안에서 돌린다. 노트북은 출력 칸을 지운 채 저장소에 올라간 그대로다.

2차 제출본과의 대응

2차 제출본(09-28)	최종(10-21)	설명
`encoder_finetune_colab_v4_0928.ipynb`	`copylane_encoder_D4-20_기능성편입_20261018.ipynb`	지금 채택된 모델을 만든 노트북으로 바꿨다
`학습결과 파일/` config · label_scheme · model.safetensors · tokenizer · tokenizer_config	같은 이름 5개	6유형 v4 모델 → 9유형 D4-20 모델

2차 제출본(09-28)	최종(10-21)	설명
`학습결과 파일/metrics.json` · `run_config.json`	`학습기록/` 의 dev 집계표 · 실험기록 발췌 · `label_scheme.json` 안의 학습 기록 칸	D4-20 은 이 두 파일을 따로 내지 않았다. 학습 설정과 dev 성능은 `label_scheme.json` (학습 행 수 · 시드별 PR-AUC · 문턱)과 실험기록에 남아 있다. 노트북이 Drive 에 쓰는 요약 파일 `results_D4-20.json` 은 저장소에 없어 넣지 못했다(공유 드라이브에서 확인 중 · 박수진)
(없음)	`모델_명세.md` · PDF · `채점_스크립트/`	새로 넣었다

주의 — `학습결과 파일/` 에는 **5개 파일만** 둔다. 앱은 폴더 안 파일 전부를 잠금과 대조하므로, 파일을 하나 더 넣거나 하위 폴더를 만들면 「잠금 밖」으로 보고 모델을 올리지 않는다(`app/model_lock.py:64-81, 183-209` · 탐색기 · 드라이브가 스스로 만드는 `desktop.ini` 등만 빼고 본다 `:41`). 그래서 기록 파일은 옆 폴더 `학습기록/` 에 두었다.

3. 베이스 모델과 구조

항목	값	근거
백본	`beomi/kcbert-base` · `BertForSequenceClassification`	`config.json` `architectures`
층 · 크기	12층 · hidden 768 · attention head 12 · FFN 3,072 · 어휘 30,000	`config.json`
위치 임베딩 길이	300(학습·추론 입력 상한은 136으로 더 짧다 — §5)	`config.json` `max_position_embeddings`
출력 층	로짓 9개 → sigmoid(유형마다 독립 확률)	`config.json` `id2label` · `app/encoder.py:424-425`
가중치 파일	`model.safetensors` 435,725,324 바이트(float32)	실측 · `config.json` `dtype`
학습 때 transformers	5.20.1(Colab)	`config.json` `transformers_version`
백본 리비전	노트북은 `MODEL_NAME = "beomi/kcbert-base"` 만 적고 리비전(커밋)을 고정하지 않았다. `label_scheme.json` 에도 이름(`backbone`)만 있고 커밋 해시는 기록되지 않았다. 2차 제출본 v4는 `0f2f3f8c…` 로 고정했었다. 다음 판부터 `backbone_revision` 칸에 고정한다(박수진 10-22)	D4-20 노트북 2절 · `label_scheme.json` · 2차 `run_config.json`

4. 라벨 스킴

9개 위반 유형을 다중 라벨로 낸다. 순서는 가중치의 로짓 순서와 같고, 앱은 `config.json` 과 `label_scheme.json` 의 순서가 다르면 모델을 올리지 않는다(`app/encoder.py:366-371`).

#	유형	학습 양성 행	문턱 — 「모름」 입력	문턱 — 품목 입력	문턱을 정한 방법
0	질병_예방치료_표방	342	0.55	0.65	목표 재현율
1	건강기능식품_오인	607	0.425	0.475	목표 재현율
2	의약품_오인	285	0.55	0.75	목표 재현율
3	거짓_과장	1,982	0.375	0.35	목표 재현율
4	소비자_기만	593	0.45	0.475	목표 재현율
5	후기_체험기_기만	195	0.55	0.55	dev F1 최대(확장 dev 양성 30 미만)
6	부당_비교광고	189	0.625	0.5	dev F1 최대(확장 dev 양성 30 미만)
7	비방광고	285	0.275	0.25	목표 재현율
8	기능성화장품_오인	23	0.25	0.40	dev F1 최대 — 확장 dev 양성 5라 **측정 불가**

- 출처: `label_scheme.json` `label_list` · `train_label_pos` · `label_thresholds` (「모름」) · `label_thresholds_by_cat` (품목) · `threshold_source` · `waiting_note` .
- 문턱 규칙(D-324 + δ 0.03): 유형·입력마다 「확장 dev 재현율이 목표 재현율 + 0.03 이상인 가장 높은 문턱」을 0.1~0.9 격자(0.025 간격)에서 고른다. 목표는 카드 17 기준선(D4-15a)의 시드 48·49·50 평균이다. 확률이 문턱 **이상**이면 후보로 낸다(`label_scheme.json` `threshold_note` · `app/encoder.py:332-334`).
- 「품목 입력」 문턱은 이름(`by_cat`)과 달리 식품·건기식·화장품이 **한 벌을 같이** 쓴다(`app/encoder.py:184-196` `LabelScheme.thresholds_for`).
- D4-20 에서 바뀐 점: 기능성화장품_오인을 「편입 대기」에서 확정 유형으로 옮겼다(D-349 보충 결정 5). 그래서 확정 유형이 8개에서 9개가 됐다.
- 학습 정답 규칙: 골든셋의 조건 M(모호) · D(판정 대상 아님) · L(적법) 행은 음성으로 다룬다. 전제가 `건기식_비인정` 인 행의 거짓_과장은 건강기능식품_오인으로 바꿔 배운다(근거에 제4호의 다른 목이 함께 있으면 거짓_과장도 남긴다 · `label_scheme.json` `target_rule` · 노트북 3절 `encoder_target`).
- 앱 계약의 위반 유형(`app/contracts.py:181` `Violation`)은 이 9개에 추천_보증_뒷광고(범위 밖)와 실증책임_위반(후보)을 더한 11개다. 인코더는 그중 9개만 낸다. 같은 곳의 설명문(`:184-186`)에 남은 「8종 · 기능성 편입 대기」는 D4-20 이전 표기다.

5. 입력과 출력

입력 — (품목, 문장) 문장쌍

항목	값	근거
형식	`[CLS] 품목 [SEP] 문장 [SEP]` · 길면 **문장 쪽만** 자른다(`truncation="only_second"`)	`label_scheme.json` `input_format` · `app/encoder.py:405-419`
품목 글자	식품 · 건기식 · 화장품 · 모름(그 밖의 품목·품목을 모를 때)	`input_format.values` · `unknown` · `app/premise.py:125-136`
최대 길이	136 토큰(넘으면 잘렸다는 표시 `truncated` 를 함께 싣는다)	`label_scheme.json` `max_len` · `app/encoder.py:411, 426`
학습 때 품목 지우기	품목이 있는 학습 행의 50 %를 배치마다 「모름」으로 바꿔 넣었다(품목을 모를 때도 쓰려고)	`input_format.cat_drop` 0.5 · 노트북 5절 `CatCollator`
전제 → 품목 대응	식품→식품 · 건기식_인정/비인정→건기식 · 화장품→화장품 · 일반상품→모름. 앱의 표(`app/premise.py`)와 다르면 로더가 멈춘다. 스킴에 없는 줄(일반식품_기능성 → 식품)은 앱의 표를 쓴다	`input_format.premise_to_input` · `app/encoder.py:233-250`

출력

- 9개 유형의 sigmoid 확률(`scores`)과, 그 입력의 문턱을 넘은 유형 목록(`candidates` · 유형 · 확률 · 문턱)이다 (`app/encoder.py:156-171`).
- 판정 그래프의 `encode` 노드는 문장마다 필요한 품목 입력 여럿(요청 품목, 분기에 쓰는 품목)을 **한 배치**로 돌려 입력마다 한 벌씩 싣는다(`app/graph.py:1124-1171` · D-339 결정 5).
- 인코더 출력은 「후보와 확신」일 뿐 그 자체로 판정이 아니다. 사전 유형이 없고 인코더 후보만 있는 문장은 2026-10-21 기준 화면에 「AI 추정 유형 있음 — 판정 아님」 표지로 나간다(`app/reasons.py:144` · D-330).
- 그래프가 사용자 답 없이 인코더 근거로 확정하는 자리는 둘이고, 둘 다 품목이 정해졌을 때만 서며 이 판 전용 확신 문턱이 걸린다: 법 축 칸이 C 인 절대형 확정 τ 0.699(`app/graph.py:3569` · `_absolute_types` · `_encoder_side`) · 전제가 답인 건강기능식품_오인 확정 τ 0.693(`app/graph.py:3592-3594` · `_hf_premise` · `_food_premise`). 두 표의 열쇠가 판 지문 `c451942436f0` 이라 되돌림 판 등 다른 판에는 이 문턱이 걸리지 않는다. 그 밖의 인코더 쪽 확정은 질문에 대한 사용자 답(사실 확인 또는 실증 여부의 「아니오」)이 있어야 선다(`_enc_confirmed` 를 부르는 나머지 자리는 모두 그 답을 함께 넘긴다 · `_enc_absolute` 는 위 절대형 자리).
- 품목을 모르는 요청에서는 기록 문장에 사실 · 실증 질문을 싣지 않고 품목을 먼저 고르게 하는 보류(고르기 대기)로 둔다. 품목별 분기 미리보기의 질문은 그대로다(2026-10-21 추가 · `app/graph.py:2937-2940, 2957-2971` `_pick_first` · D-348 보충 37 결정 2). 인코더 확률 · 문턱 · 위의 두 τ 는 바뀌지 않았다.
- 응답의 문장마다 보류의 종류(`hold_kind`)가 계산돼 실린다. 확신 부족 보류 가운데 사전 후보 · 인코더 후보 · 인코더 근거 중 하나라도 있으면 「확인 못 함」, 사전과 인코더가 둘 다 잡지 못했으면 「걸린 것 없음」이다. 어느 쪽도 통과가 아니다 (2026-10-21 추가 · `app/contracts.py:1203-1222` `hold_kind` · D-348 보충 36 결정 4).

6. 학습 데이터

항목	값	근거
골든셋	`94a79fe34363…` (sha256 앞 12자) · 8,383 행(`split` 칸: train 5,894 · test 2,489) — 노트북이 sha 와 행 수를 대조하고 다르면 멈춘다	`label_scheme.json` `golden_sha256` · `golden_rows` · 노트북 1절
학습 행 합계	**5,310 행** = 골든 학습 3,963 + 검수된 정상 합성 문구(음성) 545 + 재분할 학습(해설서) 802	`label_scheme.json` `train_rows` · `synthetic_lawful_rows` · `resplit_train_used_rows` · 노트북 4절 `EXPECT_TRAIN` (재분할 전 4,508 = 3,963 + 545) · 실험기록:1086
품목별 학습 행	식품 1,915 · 건기식 1,380 · 화장품 397 · 모름 1,618	`label_scheme.json` `train_by_cat`
정상 합성 문구	권소라 10-11 검수판 900 행 중 「적법」 334 + 「적법(사실 전제)」 211 을 학습 음성으로 · 15개 제품은 측정용으로 떼어 둠	`label_scheme.json` `synthetic_lawful_*` · 노트북 4절
학습에서 뺀 행	dev 묶음 · dev 와 같은 문구 · 조건 M · 유형 없는 위반 · 낱말 단위 행	노트북 4절
손실 가중	적법 원천(D · L) 행 손실 ×2(D4-18 ×2 레시피를 그대로 이음)	`label_scheme.json` `neg_weight` 2.0 · 실험기록:1090
dev(고르기·문턱용)	확장 dev 1,147 행 = 공통 dev 749(`4ebfcb231da703b5`) + 재분할 dev 398(`cc2d8b7e42d57266`) · 지문 `289d2bbe0c443d8e`	`label_scheme.json` `dev_*` · `resplit_dev_*`
D4-20 에서 바뀐 데이터	봉인 밖 화장품 문장 중 시행규칙 제2조 1~11호 낱말이 든 행 58(학습 50 · dev 8)을 2인 재판독(박수진 · 팀장). 기능성화장품_오인 학습 양성 10 → 23(인코더 학습 표적 기준 — 골든 학습 라벨로 세면 수가 다르다)	실험기록:1083-1087 · D-349 보충 결정 7
봉인 평가셋	골든 test 2,489 행 중 재분할(D-347)로 해설서 행 950(학습에 802 사용)과 398(dev)을 옮기고 남은 **1,141 행**(지문 `74e37657d39ec325`). **열지 않았다.** 노트북은 봉인 행을 메모리에서 바로 버린다	노트북 머리 · `label_scheme.json` `resplit_*` · `final_test = false` · `test_note`

주의 — 골든셋 · 정상 문구 CSV · 행별 확률 CSV 는 저장소와 이 묶음에 넣지 않는다(D-249). 그래서 노트북과 스크립트는 이 묶음만으로는 다시 돌릴 수 없고, 입력 파일을 공유 저장소에서 받아야 한다.

7. 학습 설정

항목	값
손실	Focal Loss(γ 1.5) · 양성 가중 α = √(음성/양성) 을 1~5로 자름
최적화	lr 2e-5 · linear 스케줄 · warmup 10 % · weight decay 0.01 · batch 32 · fp16(GPU)
회차	최대 6 epoch · early stopping patience 2 · 배포 시드의 best epoch 6
회차·시드 고르기	확장 dev 의 품목 입력 6유형(질병 · 건기식 · 의약품 · 거짓과장 · 기만 · 후기) macro PR-AUC 가 가장 높은 것(D-324 결정 3)
시드	48 · 49 · 50 → 배포 49
환경	Google Colab T4 · 시드 3번 약 13분

출처: 노트북 2절(스위치 셀) · 5절 · `label_scheme.json` `best_epoch` · `seed` · `selection_note` · 실험기록:1104.

8. 기록된 성능 (모두 dev · 게이트 아님)

8-1. 인코더 단독 — 확장 dev 6유형 macro PR-AUC

시드	품목 입력 6유형 PR-AUC
48	0.7049
49(배포)	**0.7152**
50	0.7151

출처: `label_scheme.json` `dev_macro_prauc6_by_seed`.

8-2. 학습 전에 정한 합격 기준과 결과(시드 평균 · D4-18 ×2 → D4-20)

기준	품목 입력(분기)	「모름」 입력(기록)	판정
6유형 PR-AUC(−0.02 이내)	0.7021 → 0.7117	0.6749 → 0.6883	통과
무엇이든 걸린 행 G1(−2.5 %p 이내)	440 → 445 / 482	743 → 742 / 776	통과
음성(D · L) 오판정 G3(+4 행 이내)	53.7 → 50.0	63.7 → 58.3	통과
정상 문구 171 행 후보(+5 행 이내)	2.3 → 2.3	7.3 → 6.7	통과
다른 생성기 적법 213 행(품목 입력 ≤ 29 ∧ 화장품 < 26)	20(5) → 17(6)	(보고) 59 → 47	통과
213 행이 학습에 없다	같은 문장 0		통과

출처: 실험기록:1104-1113 · `학습기록/D4-20_기능성편입_dev_20261018.md`. 측정은 박수진 PC(로컬 채점)와 Colab 확률로 했다.

- 기능성화장품_오인은 확장 dev 양성이 5개뿐이라 「측정 불가」(n < 30 · D-40)다. 참고로 시드 평균 적중 3.3 / 5(품목 입력)였다.
- 이 모델을 고르는 지표는 macro PR-AUC 다. 유형별 F1 · accuracy 는 학습 때 남기지 않아 10-22 에 따로 쟀다(8-3).

8-3. 인코더 단독 — 확장 dev 유형별 정밀도 · 재현율 · F1 (2026-10-22 재측정)

앱과 같은 로더 · 같은 문턱(`label_scheme.json` · §4)으로 확장 dev 1,147 행을 채점했다. 「모름」은 품목을 모름으로 넣은 값(「모름」 문턱), 「품목」은 골든셋의 품목을 넣은 값(품목 문턱 · 품목이 없는 행은 「모름」)이다. 정답은 골든셋의 유형 라벨이다. 문턱을 고른 같은 dev 라 낙관적이고, 판정 그래프 성적이 아니다.

유형	확장 dev 양성	정밀도 · 재현율 · F1 (모름)	정밀도 · 재현율 · F1 (품목)
질병_예방치료_표방	42	0.814 · 0.833 · 0.824	0.897 · 0.833 · 0.864
건강기능식품_오인	131	0.546 · 0.763 · 0.637	0.620 · 0.771 · 0.687
의약품_오인	37	0.463 · 0.676 · 0.549	0.458 · 0.595 · 0.518
거짓_과장	475	0.617 · 0.880 · 0.726	0.657 · 0.876 · 0.751
소비자_기만	148	0.422 · 0.547 · 0.476	0.457 · 0.581 · 0.512

유형	확장 dev 양성	정밀도 · 재현율 · F1 (모름)	정밀도 · 재현율 · F1 (품목)
후기_체험기_기만	15 (측정 불가)	0.750 · 0.800 · 0.774	0.750 · 0.800 · 0.774
부당_비교광고	25 (측정 불가)	0.526 · 0.400 · 0.455	0.545 · 0.480 · 0.511
비방광고	62	0.380 · 0.613 · 0.469	0.345 · 0.613 · 0.442
기능성화장품_오인	5 (측정 불가)	0.211 · 0.800 · 0.333	0.400 · 0.800 · 0.533

묶음 지표	「모름」	품목
macro F1 (9 유형)	0.583	0.621
macro F1 (양성 30 이상 6 유형)	0.614	0.629
micro 정밀도 · 재현율 · F1	0.555 · 0.769 · 0.645	0.591 · 0.772 · 0.669
완전 일치 accuracy(라벨 집합이 정답과 같은 행)	0.500	0.530

출처: `학습기록/D4-20_유형별지표_확장dev_20261022.md` · `.json` (박수진 · 골든 `94a79fe34363` · 판 지문 `c451942436f0`) · `채점_스크립트/d4_20_type_metrics.py`. 확장 dev 양성은 거짓_과장 475 · 소비자_기만 148 · 건강기능식품_오인 131 · 비방광고 62 · 질병_예방치료_표방 42 · 의약품_오인 37 · 부당_비교광고 25 · 후기_체험기_기만 15 · 기능성화장품_오인 5 이고, 라벨 없는 행이 285 다.

8-4. 판정 그래프에 넣고 잰 값(인코더 + 사전 + 규칙 전체)

날짜 · 측정	내용	출처
10-19 · 채택 측정 (정본 기기 · 실행 팀장)	dev 조건부 345 행(채점 221): D4-10 대비 selective risk 4.0 %(1/25) → 0.0 %(0/24) · 틀린·모호 확정 5 → 3. 판정을 내린 행이 25 · 24(D4-10 · D4-20)로 30 미만이라 측정 불가 — 참고값	원장 10-19 ①(`docs/00_사실원장.md:9784-9800`)
10-21 · 최신(정본 기기 · `01287d6`)	조건부 1,132 행(채점 783): 확정위반 75 · selective risk 1.5 % [0.3 %, 8.1 %] · 판정을 내린 행 66 · coverage 8.4 %	원장 10-21(클론 B) ⑤(`docs/00_사실원장.md:10067`)

- 두 값은 측정 범위와 코드가 다르다. 0.0 %는 10-19의 옛 측정, 1.5 %가 최신값이다.
- 같은 데이터로 OpenAI 임베딩(small · large)을 쓴 비교에서는 이 모델(KcBERT)이 PR-AUC 에서 앞섰다(실험기록 E1-03 · 10-19 · 채택 판정 아님).

9. 파일과 sha256

파일	바이트	sha256	잠금과
config.json	1,866	`91949ca0c446ef3a1dfe9a6f26fb4d08d3458af344ccd6fdc5c810df78722d9d`	일치
label_scheme.json	8,180	`29a5ab2da3744f6c2822f499f1bed2dc72d875dd34fa83e0d53ceb3e41e4c3cb`	일치
model.safetensors	435,725,324	`cec5096127ddab2d645c2dad3963e0b96bd26f88d237eb8b0f061f2480727738`	일치
tokenizer.json	721,702	`a4a9a9569bce784386a90a0528d7f38d7886ebfd9fcd392ce9308f4a26f42c32`	일치
tokenizer_config.json	352	`c842f1e4ca132ffb493921dbe10eaf61b3be45f15a4ab983ae4a42aef76192df`	일치

- 대조: 2026-10-21 이 PC에서 이 묶음의 `학습결과 파일/` 을 앱의 대조 함수(`app.model_lock.match`)에 넣었다 → 상태 「채택」 · 판 지문 `c451942436f0`. 저장소의 `models/copylane-encoder-kcbert-D4-20-기능성편입/` 도 같은 결과였다.
- 박수진이 배포한 zip(`copylane-encoder-kcbert-D4-20-기능성편입.zip` · 405,996,384 바이트 · sha256 앞 16자 `d6de8da1f1f3742f`)과도 크기·sha 가 같다(이 PC에 받은 파일 실측 · 실험기록:1119).
- tokenizer 두 파일은 D4-10 판과 같은 파일이다(sha 같음). 가중치·config·label_scheme 은 다르다(`models.lock.json`).

10. 앱이 불러오는 방법 (코드 근거)

1. **고르기** — `.env` 의 `COPYLANE_MODEL_DIR` 에 판 이름(또는 경로)을 적는다. 이름이면 `models/<이름>` 을 본다. 비어 있으면 인코더 없이 돈다(`app/encoder.py:49-67` · `app/graph.py:1139-1141`). 바꾸는 명령은 `python launcher.py setting COPYLANE_MODEL_DIR <판 이름>` 이다(`app/encoder.py:131`). 이 PC `.env:158` 은 이 판이다.
2. **잠금 대조** — 폴더의 파일 전부(sha256 · 크기)를 `models.lock.json` 의 채택·되돌림 판과 맞춰 본다. 하나라도 다르면 「잠금 밖」이고 앱은 올리지 않는다(`app/encoder.py:122-134` · `app/model_lock.py:183-209` · D-333).
3. **스킴 검사** — `label_scheme.json` 의 라벨이 앱의 유형과 같은지, 문턱 두 벌이 있는지, 입력 형식(문장쌍 · `only_second` · 품목 글자 · 전제 대응)이 앱의 표와 같은지, `max_len` 이 있는지 본다. 어긋나면 멈춘다(`app/encoder.py:205-315`).
4. **로드** — 로컬 파일만 읽는다(`local_files_only=True` · `trust_remote_code=False` · `use_safetensors=True`). config 와 스킴의 라벨 순서를 대조하고, `max_len` 이 위치 임베딩 길이를 넘지 않는지 본다(`app/encoder.py:340-377` `JudgeEncoder.__init__`). 폴더마다 한 번만 메모리에 올린다(`app/encoder.py:437-440` `encoder_at`).
5. **추론** — 판정 그래프의 `encode` 노드가 법별 노드로 갈라지기 전에 한 번 부른다(`app/graph.py:1124-1171`). 앱 서버 프로세스 안에서 PyTorch `inference_mode` 로 돈다(`app/encoder.py:424-425`).
6. **표지** — 인코더가 돈 응답의 `judged_by` 에 `+encb:c451942436f0` 이 붙어 어느 판이 판정했는지 남는다(`app/encoder.py:89, 137-148` · `app/graph.py:4321-4335`).
7. **실패할 때** — 모델을 못 올리면 경고를 한 번 남기고 규칙 판정만으로 간다(보류 쪽이라 안전). 평가 도구는 이때 멈춘다 (`app/graph.py:1131-1133, 1157-1159, 1174-1179`).

직접 불러 보기(저장소 루트 · 앱 가상환경)

```
from pathlib import Path
from app import encoder as enc

d = Path("models/copylane-encoder-kcbert-D4-20-기능성편입")   # 또는 이 묶음의 「학습결과 파일」 폴더
print(enc.checked_dir(d))          # 상태 「채택」 · 판 지문 c451942436f0
m = enc.encoder_at(d)
p = m.predict("광고 문장", "화장품")  # 품목: 식품 · 건기식 · 화장품 · 모름
print(p.scores)                    # 9유형 확률
print(p.candidates)                # 문턱을 넘은 유형
```

2026-10-21 이 PC(CPU · transformers 5.17.0 · torch 2.14.0)에서 위 순서로 잠금 대조 「채택」, 로드, 「모름」·「화장품」 두 입력 추론이 오류 없이 도는 것을 확인했다. 출력 형식만 확인한 것이고 성능 측정이 아니다.

11. 판 이력

판	날짜	바뀐 것	상태
v4(2차 제출본 · 소성민)	09-28	확정 6유형 · 골든 09-25 판(8,432 행) · 문장만 입력	대체됨
v10-합성O-이유O	~10-07	—	잠금 되돌림 3순위 (D-331)
v10-카드17-검수정상 음성	10-13	검수한 정상 카피를 음성으로 재학습	잠금 되돌림 2순위 (D-331)
D4-10 품목입력사실전제	10-13~15	(품목, 문장) 쌍 입력 · 판 지문 `ef2c12a3b90f`	잠금 되돌림 1순위 (D-339)
D4-18 적법가중 ×2	10-16	적법 원천 행 손실 ×2	교체 후보 · 잠금되지 않음
D4-20 기능성편입	학습 10-18 · 채택 10-19	×2 에서 출발 · 기능성화장품_오인 편입 · 재판독 골든 `94a79fe34363`	**잠금 채택(D-349)**

출처: `models.lock.json` · 결정기록 D-331 · D-339 · D-349 · `docs/psj/reports/D4_실험기록_20261012.md` .

12. 함께 쓰이는 다른 모델 (목록만)

모델	종류	이 묶음에 넣었나
`nlpai-lab/KURE-v1`	검색 임베딩(사전학습 · 1024차원 · 팀이 학습하지 않음)	넣지 않음 — 앱의 판정 경로는 이 기기의 Hugging Face 캐시에서만 올린다(`local_files_only` · `app/retrieve.py:391-414` `encode` · `app/settings.py:351-381` `load_kwargs` · 허용 목록 `:186`). 허브에서 받는 것은 임베딩 도구(`scripts/embed.py` · `launcher.py embed`)뿐이다. 캐시가 없으면 판정은 돌고 조문 검색만 어휘로 한다. 리비전은 고정하지 않았다(`embed_revision` 빈 값)
sLLM 고쳐 쓰기 어댑터 `copylane_sllm_lora_adapter_q4b-v14`(1단계) · `copylane_sllm_persona_adapter_q4b_polish`(잠금 2단계 칸)	Qwen3-4B-Instruct-2507 + LoRA(이서은) · 잠금 채택 D-335	넣지 않음 — 「자체 sLLM」 산출물에서 따로 낸다. 2단계 다듬기의 기본은 규칙판으로 하고 이 어댑터는 되돌림 · 고객층 말투 전용으로 정했다(D-342 · 10-15 결정 · 붙인 조건은 집행 대기)
리랭커	—	없음(미구현)

13. 한계와 남은 일

- 모든 성능 수치가 dev 값이다. 이 모델은 봉인 평가셋에서 잰 적이 없고, 재분할 뒤의 새 봉인 평가셋 1,141 행은 봉인 회차 전까지 열지 않는다(D-347). 재분할 전의 옛 봉인 2,489 행은 10-14 에 이전 판(v10 · D4-10)을 인코더 단독으로 한 번 쟀을 뿐이다(D4-T · 보고 전용).
- 기능성화장품_오인은 dev 양성이 5개라 성능을 말할 수 없다(측정 불가). 이 유형은 법 축 칸이 A 라 그래프가 답 없이 확정하지 않고 기능성화장품 심사 여부를 묻는 질문으로 보낸다(「아니오」 답이면 확정 · `app/graph.py` `_cosmetic_function` · `label_scheme.json` `waiting_note`).
- 알려진 오탐 꼴: 화장품 문장의 의약품_오인 · 식품 후기 꼴 문장의 후기_체험기_기만(실험기록 D4-20 결과 · D-349 결정 4 「모델 교체는 인코더 쪽 절대형 오탐의 해법이 아니다」).
- `label_scheme.json` 안의 `adoption.verdict` 는 「팀장 대조 · 채택 측정 대기」, `loader_note` 는 「잠금에 올리기 전에는 앱이 올리지 않는다」로 남아 있다. 배포 폴더를 만들 때(10-18) 적은 글이고, 실제로는 10-19에 잠금 채택됐다. 파일을 고치면 sha 가 바뀌어 잠금 밖이 되므로 그대로 두고, 다음 판에서 고친다(박수진 10-22). 채택 사실의 정본은 `models.lock.json` 과 결정기록이다.
- 백본 리비전이 고정되지 않았고 기록도 없다(§3) — 다음 판부터 고정한다.
- D-349 에 남은 확인 칸(×2 판의 sha 대조 · 세 유형 문턱)은 아직 비어 있다(결정기록 D-349 「정하지 않은 것」).